AG3 - Actividad Guiada 3

Nombre: Wilson Bello V.
Link: https://drive.google.com/drive/folders/1pXG7o4RtxqaONwFa3zGyzJCySFHfHJM9?usp=sharing
Github: https://github.com/whbello/Algoritmos-de-optimizaci-n.git

Carga de librerias

Las librerias propuesta al ejecutarlas en el entorno de Google Copab, presentaban conflictos de dependencias. La opción implementada fue trabajar el archivo en forma local, se creo un entorno virtual y allí se instalaron algunas librerias mas actualizadas que no generaron conflictos y se pudo presentar la mejora a la función `genera_vecina_aleatorio`. Se creo archivo requirements_AG3.txt, para consulta y verificación de librerias utilizadas.

```
"""
Estas son las librerias iniciales del Notebook pero al proceder con su corrida pre
de conflictos de dependencias
"""

!pip install requests
!pip install tabulate>=0.9 networkx>=3.0
!pip install tsplib95 --no-deps
!pip install deprecated
```

```
Collecting requests
  Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Using cached charset_normalizer-3.4.4-cp313-cp313-win_amd64.whl.metadata (38 kB)
Collecting idna<4,>=2.5 (from requests)
  Using cached idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,>=1.21.1 (from requests)
  Using cached urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2017.4.17 (from requests)
  Downloading certifi-2026.2.25-py3-none-any.whl.metadata (2.5 kB)
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
Downloading certifi-2026.2.25-py3-none-any.whl (153 kB)
Using cached charset_normalizer-3.4.4-cp313-cp313-win_amd64.whl (107 kB)
Using cached idna-3.11-py3-none-any.whl (71 kB)
Using cached urllib3-2.6.3-py3-none-any.whl (131 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
```

```
Successfully installed certifi-2026.2.25 charset_normalizer-3.4.4 idna-3.11 request

[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
Collecting tsplib95
  Using cached tsplib95-0.7.1-py2.py3-none-any.whl.metadata (6.3 kB)
Using cached tsplib95-0.7.1-py2.py3-none-any.whl (25 kB)
Installing collected packages: tsplib95
Successfully installed tsplib95-0.7.1

[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
Collecting deprecated
  Using cached deprecated-1.3.1-py2.py3-none-any.whl.metadata (5.9 kB)
Collecting wrapt<3,>=1.10 (from deprecated)
  Downloading wrapt-2.1.1-cp313-cp313-win_amd64.whl.metadata (7.6 kB)
Using cached deprecated-1.3.1-py2.py3-none-any.whl (11 kB)
Downloading wrapt-2.1.1-cp313-cp313-win_amd64.whl (60 kB)
Installing collected packages: wrapt, deprecated
Successfully installed deprecated-1.3.1 wrapt-2.1.1
ERROR: pip's dependency resolver does not currently take into account all the packa
tsplib95 0.7.1 requires Click>=6.0, which is not installed.
tsplib95 0.7.1 requires Deprecated~=1.2.9, but you have deprecated 1.3.1 which is i
tsplib95 0.7.1 requires networkx~=2.1, but you have networkx 3.6.1 which is incompa
tsplib95 0.7.1 requires tabulate~=0.8.7, but you have tabulate 0.9.0 which is incom

[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
!pip install "deprecated==1.2.18" "networkx==2.8.8" "tabulate==0.8.10" click --for
```

```
[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
import tsplib95
print(tsplib95.__version__)
```

```
0.7.1
```

```
# Ya se tiene instalada la versión correcta deprecated==1.2.18 y si se ejecuta est
# pip instalará la 1.3.1 que es incompatible con tsplib95 y se volverá a tener con

!pip install deprecated
```

Carga de los datos del problema

```
import urllib.request #Hacer llamadas http a paginas de la red
import tsplib95       #Modulo para las instancias del problema del TSP
import math           #Modulo de funciones matematicas. Se usa para exp
import random         #Para generar valores aleatorios


#Descargamos el fichero de datos(Matriz de distancias)
file = "swiss42.tsp" ;
#urllib.request.urlretrieve("https://raw.githubusercontent.com/mastqe/tsplib/refs/
urllib.request.urlretrieve("https://raw.githubusercontent.com/mastqe/tsplib/refs/h
#!gzip -d swiss42.tsp.gz     #Descomprimir el fichero de datos

#Coordendas 51-city problem (Christofides/Eilon)
#file = "eil51.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg

#Coordenadas - 48 capitals of the US (Padberg/Rinaldi)
#file = "att48.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg


#http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95
#Documentacion :
  # http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf
  # https://tsplib95.readthedocs.io/en/stable/pages/usage.html
  # https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
  # https://pypi.org/project/tsplib95/
```

```
('swiss42.tsp', <http.client.HTTPMessage at 0x1a9a2581bf0>)
```

```
#Carga de datos y generación de objeto problem
##############################################################################
problem = tsplib95.load(file)

#Nodos
Nodos = list(problem.get_nodes())

#Aristas
Aristas = list(problem.get_edges())
```

```
Nodos
```

```
[0,
 1,
 2,
 3,
 4,
 5,
 6,
 7,
 8,
 9,
 10,
 11,
 12,
 13,
 14,
 15,
 16,
 17,
 18,
 19,
 20,
 21,
 22,
 23,
 24,
 25,
 26,
 27,
 28,
 29,
 30,
 31,
 32,
 33,
 34,
 35,
 36,
 37,
 38,
 39,
 40,
 41]
```

```
Aristas
# https://github.com/mastqe/tsplib/blob/master/swiss42.tsp
```

```
[(0, 0),
 (0, 1),
 (0, 2),
 (0, 3),
 (0, 4),
 (0, 5),
 (0, 6),
```

```
      (0, 7),
      (0, 8),
      (0, 9),
      (0, 10),
      (0, 11),
      (0, 12),
      (0, 13),
      (0, 14),
      (0, 15),
      (0, 16),
      (0, 17),
      (0, 18),
      (0, 19),
      (0, 20),
      (0, 21),
      (0, 22),
      (0, 23),
      (0, 24),
      (0, 25),
      (0, 26),
      (0, 27),
      (0, 28),
      (0, 29),
      (0, 30),
      (0, 31),
      (0, 32),
      (0, 33),
      (0, 34),
      (0, 35),
      (0, 36),
      (0, 37),
      (0, 38),
      (0, 39),
      (0, 40),
      (0, 41),
      (1, 0),
      (1, 1),
      (1, 2),
      (1, 3),
      (1, 4),
      (1, 5),
      (1, 6),
      (1, 7),
      (1, 8),
      (1, 9),
      (1, 10),
      (1, 11),
      (1, 12),
      (1, 13),
      (1, 14),
```

```
NOMBRE: swiss42
TIPO: TSP
COMENTARIO: 42 Staedte Schweiz (Fricker)
DIMENSION: 42
EDGE_WEIGHT_TYPE: EXPLICIT
EDGE_WEIGHT_FORMAT: FULL_MATRIX
EDGE_WEIGHT_SECTION
0 15 30 23 32 55 33 37 92 114 92 110 96 90 74 76 82 72 78 82 159 122 131 206 112 57 28 43 70
  15 0 34 23 27 40 19 32 93 117 88 100 87 75 63 67 71 69 62 63 96 164 132 131 212 106 44 33 5
  30 34 0 11 18 57 36 65 62 84 64 89 76 93 95 100 104 98 57 88 99 130 100 101 179 86 51 4 18
  23 23 11 0 11 48 26 54 70 94 69 75 75 84 84 89 92 89 54 78 99 141 111 109 89 89 11 11 11 54
  32 27 18 11 0 40 20 58 67 92 61 78 65 76 83 89 91 95 43 72 110 141 116 105 190 81 34 19 35
  55 40 57 48 40 0 23 55 96 123 78 75 36 36 66 66 63 95 34 34 137 174 156 129 224 90 15 59 75
  33 19 36 26 20 23 0 45 85 111 75 82 69 60 63 70 71 85 44 52 115 161 136 122 210 91 25 37 54
  37 32 65 54 58 55 45 0 124 149 118 126 113 80 42 42 40 40 87 87 94 158 158 163 242 135 65 6
  92 93 62 70 67 96 85 124 0 28 29 68 63 122 148 155 156 159 67 129 148 78 80 39 129 46 82 65
 114 117 84 94 92 123 111 149 28 0 54 91 88 150 174 181 182 181 95 157 159 50 65 27 102 65 11
  92 88 64 69 61 78 75 118 29 54 0 39 34 99 134 142 141 157 44 110 161 103 109 52 154 22 63 6
 110 100 89 89 78 75 82 126 68 91 39 0 14 80 129 139 135 167 39 98 187 136 148 81 186 28 61 9
  96 87 76 75 65 62 69 113 63 88 34 14 0 72 117 128 124 153 26 88 174 136 142 82 187 32 48 79
  90 75 93 84 76 36 60 80 122 150 99 80 72 0 59 71 63 116 56 25 170 201 189 151 252 104 44 95
  74 63 95 84 83 56 63 42 148 174 134 129 117 59 0 11 8 63 93 35 135 223 195 184 273 146 71 9
```

```
#Probamos algunas funciones del objeto problem

#Distancia entre nodos
problem.get_weight(3, 0)

#Todas las funciones
#Documentación: https://tsplib95.readthedocs.io/en/v0.6.1/modules.html

#dir(problem)
```

```
23
```

Funciones basicas

```
##############################################################################
#Funcionas basicas
##############################################################################

#Se genera una solucion aleatoria con comienzo en en el nodo 0
def crear_solucion(Nodos):
  solucion = [Nodos[0]]
  for n in Nodos[1:]:
    solucion = solucion + [random.choice(list(set(Nodos) - set({Nodos[0]}) - set(s
  return solucion

#Devuelve la distancia entre dos nodos
def distancia(a,b, problem):
```

```
  return problem.get_weight(a,b)

#Devuelve la distancia total de una trayectoria/solucion
def distancia_total(solucion, problem):
  distancia_total = 0
  for i in range(len(solucion)-1):
    distancia_total += distancia(solucion[i] ,solucion[i+1] ,  problem)
  return distancia_total + distancia(solucion[len(solucion)-1] ,solucion[0], probl

sol_temporal = crear_solucion(Nodos)


print(sol_temporal   )
print(distancia_total(sol_temporal, problem))
```

```
[0, 4, 17, 7, 1, 12, 35, 41, 2, 36, 40, 8, 11, 38, 23, 5, 9, 26, 37, 27, 24, 29, 16
5022
```

BUSQUEDA ALEATORIA

```
##############################################################################
# BUSQUEDA ALEATORIA
##############################################################################

def busqueda_aleatoria(problem, N):
  #N es el numero de iteraciones
  Nodos = list(problem.get_nodes())

  mejor_solucion = []
  #mejor_distancia = 10e100                        #Inicializamos con un valor al
  mejor_distancia = float('inf')                   #Inicializamos con un valor al

  for i in range(N):                               #Criterio de parada: repetir N
    solucion = crear_solucion(Nodos)               #Genera una solucion aleatoria
    distancia = distancia_total(solucion, problem)  #Calcula el valor objetivo(dis

    if distancia < mejor_distancia:                #Compara con la mejor obtenida
      mejor_solucion = solucion
      mejor_distancia = distancia


  print("Mejor solución:" , mejor_solucion)
  print("Distancia     :" , mejor_distancia)
  return mejor_solucion
```

```
#Busqueda aleatoria con 5000 iteraciones
solucion = busqueda_aleatoria(problem, 50000)
```

```
Mejor solución: [0, 32, 4, 27, 5, 30, 24, 22, 31, 1, 28, 33, 34, 17, 35, 36, 29, 20
Distancia     : 3575
```

BUSQUEDA LOCAL

```
##############################################################################
# BUSQUEDA LOCAL(1 paso)
##############################################################################
def genera_vecina(solucion):
  #Generador de soluciones vecinas: 2-opt (intercambiar 2 nodos) Si hay N nodos se
  #Se puede modificar para aplicar otros generadores distintos que 2-opt
  #print(solucion)
  mejor_solucion = []
  mejor_distancia = 10e100
  for i in range(1,len(solucion)-1):          #Recorremos todos los nodos en bucle
    for j in range(i+1, len(solucion)):

      #Se genera una nueva solución intercambiando los dos nodos i,j:
      #  (usamos el operador + que para listas en python las concatena) : ej.: [1,
      vecina = solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + so

      #Se evalua la nueva solución ...
      distancia_vecina = distancia_total(vecina, problem)

      #... para guardarla si mejora las anteriores
      if distancia_vecina <= mejor_distancia:
        mejor_distancia = distancia_vecina
        mejor_solucion = vecina
  return mejor_solucion


#solucion = [1, 47, 13, 41, 40, 19, 42, 44, 37, 5, 22, 28, 3, 2, 29, 21, 50, 34, 3
print("Distancia Solucion Incial:" , distancia_total(solucion, problem))


nueva_solucion = genera_vecina(solucion)
print("Distancia Mejor Solucion Local:", distancia_total(nueva_solucion, problem))
```

```
Distancia Solucion Incial: 3575
Distancia Mejor Solucion Local: 3268
```

```
#Busqueda Local(iteraciones):
#  - Sobre el operador de vecindad 2-opt(funcion genera_vecina)
```

```
#  - Sin criterio de parada, se para cuando no es posible mejorar.
def busqueda_local(solucion, problem):
  mejor_solucion = []

  #Generar una solucion inicial de referencia(aleatoria)
  #solucion_referencia = crear_solucion(Nodos)
  solucion_referencia = solucion
  mejor_distancia = distancia_total(solucion_referencia, problem)

  iteracion=0             #Un contador para saber las iteraciones que hacemos
  while(1):
    iteracion +=1         #Incrementamos el contador
    #print('#',iteracion)

    #Obtenemos la mejor vecina ...
    vecina = genera_vecina(solucion_referencia)

    #... y la evaluamos para ver si mejoramos respecto a lo encontrado hasta el mo
    distancia_vecina = distancia_total(vecina, problem)

    #Si no mejoramos hay que terminar. Hemos llegado a un minimo local(según nuest
    if distancia_vecina < mejor_distancia:
      #mejor_solucion = copy.deepcopy(vecina)   #Con copia profunda. Las copias en
      mejor_solucion = vecina                   #Guarda la mejor solución encontra
      mejor_distancia = distancia_vecina

    else:
      print("En la iteracion ", iteracion, ", la mejor solución encontrada es:" ,
      print("Distancia     :" , mejor_distancia)
      return mejor_solucion

    solucion_referencia = vecina


sol = busqueda_local(nueva_solucion, problem )
```

```
En la iteracion  28 , la mejor solución encontrada es: [0, 3, 4, 2, 27, 28, 29, 22,
Distancia     : 1634
```

SIMULATED ANNEALING

Propuesta de mejora a función `genera_vecina_aleatorio`

La función actual selecciona dos nodos aleatorios y los intercambia de posición.
Ejemplo:
Si la solución original es: [0, 1, 2, 3, 4, 5, 0]

Se eligen los nodos: i=2, j=4
El Segmento invertido queda: [2, 3, 4] -> [4, 3, 2]
Tenemos como resultado: [0, 1, 4, 3, 2, 5, 0]

Solo se intercambio los nodos de las posiciones `i` y `j`, dejando intacto el resto del recorrido. Este movimiento es limitado porque no considera si el intercambio genera o limita cruces en la ruta.

El intercambio de nodos puede generar rutas con `cruces geometricos`, lo que matemáticamente implica recorridos mas largos. Un cruce se presenta cuando dos aristas del recorrido se intersectan.
Ejemplo:

Si la solución original es: [0, 1, 2, 3, 4, 5, 0]
Y se eligen los nodos: i=2, j=4
El Segmento invertido queda: [2, 3, 4] -> [4, 3, 2]
Tenemos como resultado: [0, 1, 4, 3, 2, 5, 0]

Solo se intercambian los nodos en las posiciones `i` y `j`, dejando intacto el resto del recorrido. Este movimiento es limitado porque no considera si el intercambio genera o elimina cruces en la ruta.

El intercambio simple puede generar rutas con cruces geómetricos, lo que matematicamente implica recorridos mas largos. Un cruce ocurre cuando dos aristas del recorrido se intersectan, como en el siguiente ejemplo:

con cruce sin cruce

1 ----> 3 1 <----- 3

2 <---- 4 2 -----> 4

Limitación Fundamental

El intercambio simple opera de forma **ciega sobre la topologia del recorrido**. No considera si las aristas resultantes generan o eliminan cruces geométricos en la ruta. Un cruce ocurre cuando dos aristas del recorrido se intersectan en el plano.
El cambio simple **no garantiza eliminar estos cruces** y puede incluso generarlos, produciendo soluciones vecinas de peor calidad estructural.

Función Mejorada: `Movimiento 2-opt Real`

Al utilizar el movimiento **2-opt** propuesto por Cross, metodo de mejora local mas utilizado en el TSP. Su principio es simple pero poderoso: si dos aristas del recorrido se cruzan,

siempre es posible reconectarlas de forma que el cruce desaparezca y la distancia total disminuya o se mantenga igual.

¿Que hace?: En lugar de intercambiar dos nodos, el movimiento 2-opt **elimina dos aristas del recorrido actual y las reemplaza por dos aristas nuevas**, lo que equivale a invertir el segmento completo entre las dos posiciones seleccionadas.

Ejemplo con 6 nodos:

Si la solución original es: [0, 1, 2, 3, 4, 5, 0]
Y se eligen los nodos: i=2, j=4
El Segmento invertido queda: [2, 3, 4] -> [4, 3, 2]
Tenemos como resultado: [0, 1, 4, 3, 2, 5, 0]

¿Por qué 2-opt es superior?

La inversión del segmento equivale a **eliminar dos aristas del recorrido actual y reemplazarlas por dos aristas nuevas**, lo que garantiza que si la nueva solución es mejor, se ha eliminado por lo menos un cruce geométrico.

Esta mejora permite al algoritmo explorar vecindades de mayor calidad en cada iteración, lo que se traduce en una convergencia más eficiente hacia soluciones con menor distancia total, sin ningun costo computacional adicional, ya que ambas operaciones tiene complejidad **O(1)**.

Referencias

- Croes, G.A. (1958). *A method for solving traveling-salesman problems*. Operations Research, 6(6), 791-812.
- Lin, S. (1965). *Computer solutions of the traveling salesman problem*. Bell System Technical Journal, 44(10), 2245-2269.


```
################################################################################
# SIMULATED ANNEALING
################################################################################

#Generador de 1 solucion vecina 2-opt 100% aleatoria (intercambiar 2 nodos)
#Mejorable eligiendo otra forma de elegir una vecina.
"""def genera_vecina_aleatorio(solucion):

  #Se eligen dos nodos aleatoriamente
  i,j = sorted(random.sample( range(1,len(solucion)) , 2))

  #Devuelve una nueva solución pero intercambiando los dos nodos elegidos al azar
  return solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion
"""
```

```
def genera_vecina_2opt(solucion):
    i, j = sorted(random.sample(range(1, len(solucion)), 2))
    return solucion[:i] + solucion[i:j+1][::-1] + solucion[j+1:]

#Funcion de probabilidad para aceptar peores soluciones
def probabilidad(T,d):
  if random.random() <  math.exp( -1*d / T)  :
    return True
  else:
    return False

#Funcion de descenso de temperatura
def bajar_temperatura(T):
  return T*0.99
```

```
def recocido_simulado(problem, TEMPERATURA ):
  #problem = datos del problema
  #T = Temperatura

  solucion_referencia = crear_solucion(Nodos)
  distancia_referencia = distancia_total(solucion_referencia, problem)

  mejor_solucion = []             #x* del seudocodigo
  mejor_distancia = 10e100        #F* del seudocodigo


  N=0
  while TEMPERATURA > .0001:
    N+=1
    #Genera una solución vecina
    #vecina =genera_vecina_aleatorio(solucion_referencia)
    vecina =genera_vecina_2opt(solucion_referencia)

    #Calcula su valor(distancia)
    distancia_vecina = distancia_total(vecina, problem)

    #Si es la mejor solución de todas se guarda(siempre!!!)
    if distancia_vecina < mejor_distancia:
        mejor_solucion = vecina
        mejor_distancia = distancia_vecina

    #Si la nueva vecina es mejor se cambia
    #Si es peor se cambia según una probabilidad que depende de T y delta(distanci
    if distancia_vecina < distancia_referencia or probabilidad(TEMPERATURA, abs(di
      #solucion_referencia = copy.deepcopy(vecina)
      solucion_referencia = vecina
      distancia_referencia = distancia_vecina
```

```
    #Bajamos la temperatura
    TEMPERATURA = bajar_temperatura(TEMPERATURA)

  print("La mejor solución encontrada es " , end="")
  print(mejor_solucion)
  print("con una distancia total de " , end="")
  print(mejor_distancia)
  return mejor_solucion

sol  = recocido_simulado(problem, 10000000)
```

```
La mejor solución encontrada es [0, 1, 7, 5, 6, 4, 27, 3, 2, 28, 30, 34, 33, 38, 22
con una distancia total de 1593
```

```
!pip install numpy matplotlib
```

```
Requirement already satisfied: numpy in d:\work\cursos\ia\u_valencia_master_ia\03.
Requirement already satisfied: matplotlib in d:\work\cursos\ia\u_valencia_master_ia
Requirement already satisfied: contourpy>=1.0.1 in d:\work\cursos\ia\u_valencia_mas
Requirement already satisfied: cycler>=0.10 in d:\work\cursos\ia\u_valencia_master_
Requirement already satisfied: fonttools>=4.22.0 in d:\work\cursos\ia\u_valencia_ma
Requirement already satisfied: kiwisolver>=1.3.1 in d:\work\cursos\ia\u_valencia_ma
Requirement already satisfied: packaging>=20.0 in d:\work\cursos\ia\u_valencia_mast
Requirement already satisfied: pillow>=8 in d:\work\cursos\ia\u_valencia_master_ia\
Requirement already satisfied: pyparsing>=3 in d:\work\cursos\ia\u_valencia_master_
Requirement already satisfied: python-dateutil>=2.7 in d:\work\cursos\ia\u_valencia
Requirement already satisfied: six>=1.5 in d:\work\cursos\ia\u_valencia_master_ia\0

[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
# Esta libreria contiene herramientas para clasificación, regresión, clustering, y
!pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in d:\work\cursos\ia\u_valencia_master_
Requirement already satisfied: numpy>=1.24.1 in d:\work\cursos\ia\u_valencia_master
Requirement already satisfied: scipy>=1.10.0 in d:\work\cursos\ia\u_valencia_master
Requirement already satisfied: joblib>=1.3.0 in d:\work\cursos\ia\u_valencia_master
Requirement already satisfied: threadpoolctl>=3.2.0 in d:\work\cursos\ia\u_valencia

[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Representación en un grafo a partir de la matriz de distancias(Optimización de posiciones usando escalado multidimensional (MDS)

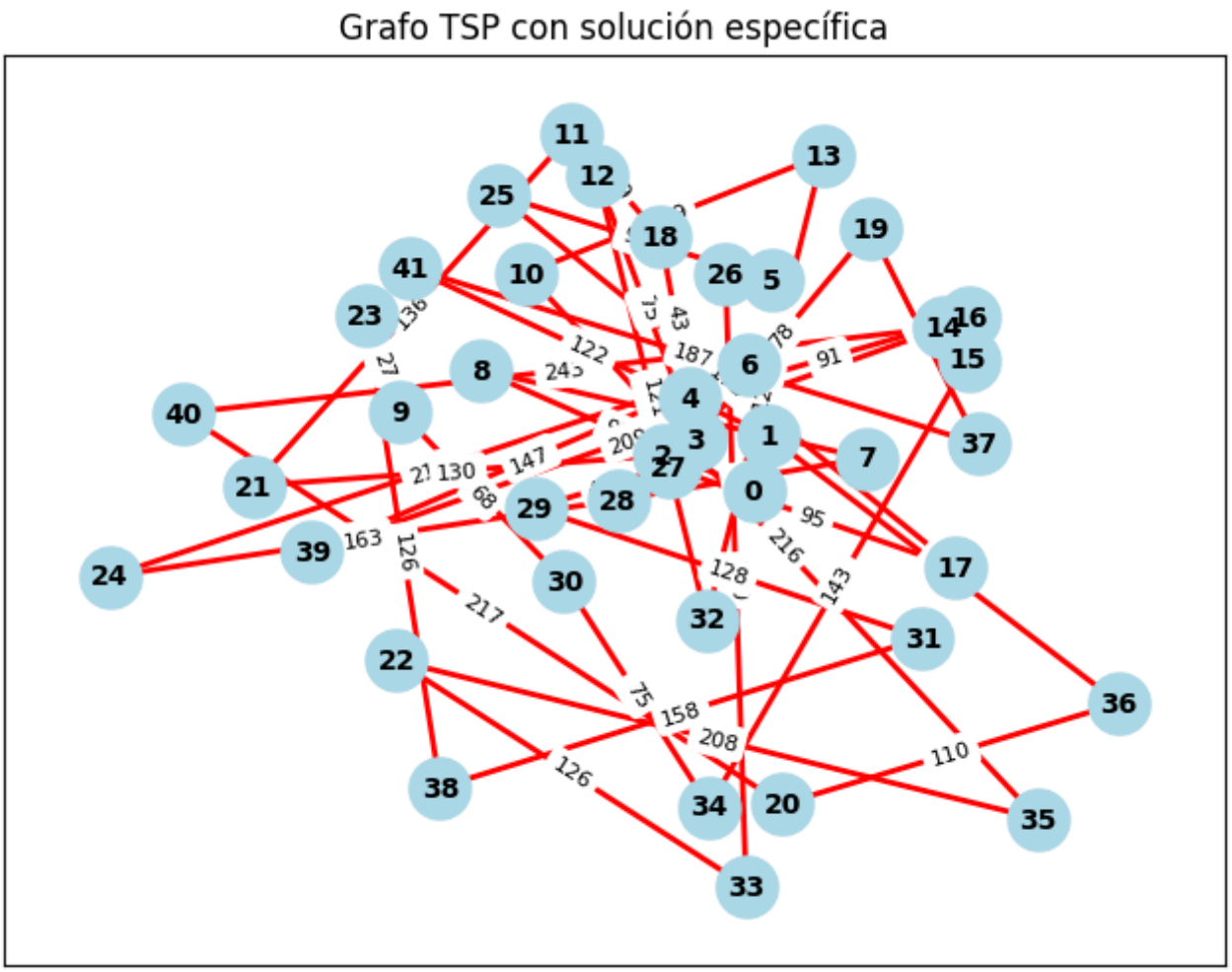
Multidimensional scaling problem(MDS):
https://en.wikipedia.org/wiki/Multidimensional_scaling

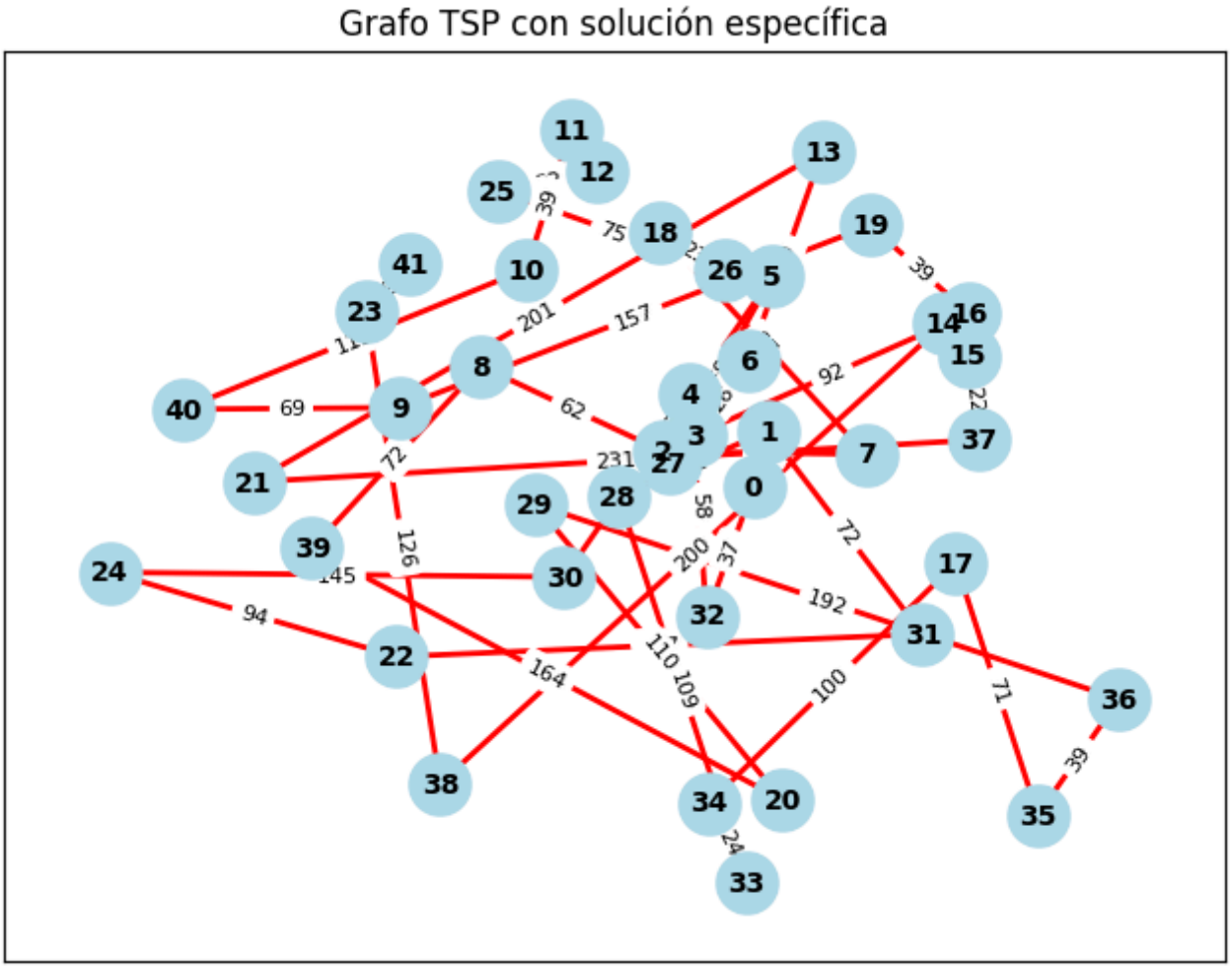
```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.manifold import MDS  # Multidimensional Scaling o Escalado Multidimen

def plot_tsp_solution(distance_matrix, tsp_solution):
    """
    Dibuja el grafo de un TSP con las posiciones calculadas mediante MDS y muestra
    solo las aristas correspondientes a la solución del TSP.

    :param distance_matrix: np.ndarray, matriz de distancias entre nodos
    :param tsp_solution: list, lista de nodos en el orden de la solución del TSP
    """
    # Crear el grafo completo
    G = nx.Graph()
    num_nodes = len(distance_matrix)
    for i in range(num_nodes):
        for j in range(i + 1, num_nodes):
            G.add_edge(i, j, weight=distance_matrix[i][j])

    # Usar MDS para calcular posiciones de los nodos
    mds = MDS(n_components=2, dissimilarity="precomputed", random_state=42)
    positions = mds.fit_transform(distance_matrix)

    # Convertir las posiciones en un diccionario para networkx
    pos = {i: positions[i] for i in range(num_nodes)}

    # Crear un subgrafo con las aristas del camino TSP
    TSP_G = nx.Graph()
    for i in range(len(tsp_solution) - 1):
        u = tsp_solution[i]
        v = tsp_solution[i + 1]
        TSP_G.add_edge(u, v, weight=distance_matrix[u][v])

    # Dibujar el grafo
    plt.figure(figsize=(8, 6))

    # Dibujar nodos
```

```
    nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=500)

    # Dibujar las aristas del camino TSP
    nx.draw_networkx_edges(TSP_G, pos, edge_color='red', width=2)

    # Añadir etiquetas a los nodos y pesos de las aristas
    nx.draw_networkx_labels(G, pos, font_size=10, font_weight='bold')
    edge_labels = nx.get_edge_attributes(TSP_G, 'weight')
    nx.draw_networkx_edge_labels(TSP_G, pos, edge_labels=edge_labels, font_size=8)

    plt.title("Grafo TSP con solución específica")
    plt.show()
```

```
plot_tsp_solution(problem.edge_weights, crear_solucion(Nodos))
```

```
d:\Work\Cursos\IA\U_Valencia_Master_IA\03. Algoritmos de Optimización\Actividad Gui
  warnings.warn(
d:\Work\Cursos\IA\U_Valencia_Master_IA\03. Algoritmos de Optimización\Actividad Gui
  warnings.warn(
d:\Work\Cursos\IA\U_Valencia_Master_IA\03. Algoritmos de Optimización\Actividad Gui
  warnings.warn(
```



```
plot_tsp_solution(problem.edge_weights, solucion)
```

```
d:\Work\Cursos\IA\U_Valencia_Master_IA\03. Algoritmos de Optimización\Actividad Gui
  warnings.warn(
d:\Work\Cursos\IA\U_Valencia_Master_IA\03. Algoritmos de Optimización\Actividad Gui
  warnings.warn(
d:\Work\Cursos\IA\U_Valencia_Master_IA\03. Algoritmos de Optimización\Actividad Gui
  warnings.warn(
```